ESCUELA SUPERIOR POLITECNICA DEL CHIMBORAZO

FACULTAD DE INFORMATICA Y ELECTRONICA

APLICACIONES INFORMATICAS II

Arquitectura del sistema

DESARROLLO DE UN SISTEMA WEB Y MÓVIL PARA LA GESTION
INTEGRAL EN LA EMPRESA
ICAROCONSTRUCTORES BMGM S.A.S

Ivan Santiago Pulgar Leon
Isaac Sebastián Castro Muesmueran

Ingeniero:

JULIO R.SANTILLÁN C.

Riobamba 7/04/2026

Contenido

1. Contexto y Requisitos 3
1.1 Descripción del Sistema 3
1.2 Atributos de Calidad (Drivers Arquitectónicos) 3
2. Arquitectura Seleccionada 4
2.1 Estilo Arquitectónico Principal: Layered Monolith con API REST 4
3. Diagrama de Capas y Flujo de una Petición 6
4. Diagrama Arquitectónico Completo (Componentes y Despliegue) 7
5. Patrones Arquitectónicos Adoptados 7
5.1 Patrón: Layered Architecture (Capas) 7
5.2 Patrón: RBAC — Role-Based Access Control 8
5.3 Patrón: JWT Stateless Authentication 9
5.4 Patrón: Audit Log Inmutable (Append-Only) 9
5.5 Patrón: API-First (REST con versionamiento) 10
5.6 Selección de Base de Datos: PostgreSQL 17 10
5.7 Selección de ORM: Prisma v5 14
6. Patrón de Despliegue: Docker Compose 15
7. Frontend: PWA con React + Vite 16
Justificación del Stack 16
Patrón: Context API + Axios Interceptors 16
9. Estructura de Archivos del Proyecto 17
10. Decisiones Arquitectónicas Rechazadas 17

1. Contexto y Requisitos

1.1 Descripción del Sistema

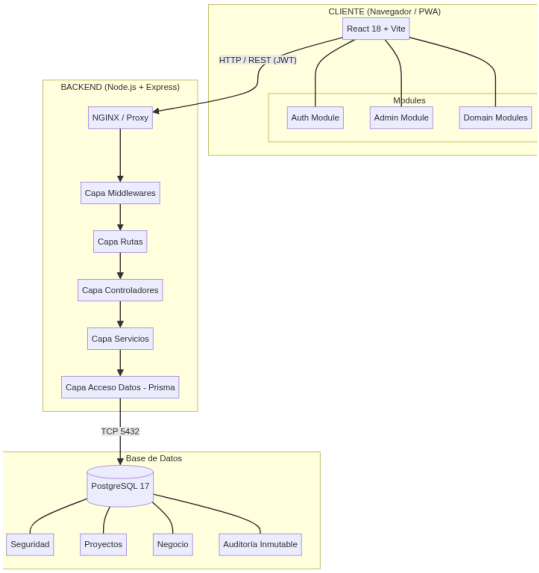
ICARO es un ERP (Enterprise Resource Planning) corporativo especializado para empresas constructoras. Gestiona:

- Control de avance de obra por rubros presupuestados
- Requerimientos y procesos de compra
- Inventario de materiales por proyecto
- Cierres contables mensuales con trazabilidad
- Seguridad multi-rol con auditoría completa

1.2 Atributos de Calidad (Drivers Arquitectónicos)

Atributo	**Prioridad**	**Implicación**
Seguridad	Crítica	Datos financieros y contractuales sensibles. Multi-rol RBAC obligatorio.
Trazabilidad	Crítica	Auditoría inmutable de cada CUD. Requisito legal/contractual.
Disponibilidad	Alta	Operación en campo (obra) puede ser intermitente. Necesita modo offline.
Escalabilidad horizontal	Media	Múltiples proyectos simultáneos. Crece por proyecto, no por carga masiva.
Mantenibilidad	Alta	Equipo pequeño. El código debe ser simple de extender módulo a módulo.
Performance	Media	Usuarios concurrentes: < 50. No es un sistema de alta concurrencia.

2. Arquitectura Seleccionada



2.1 Estilo Arquitectónico Principal: API REST

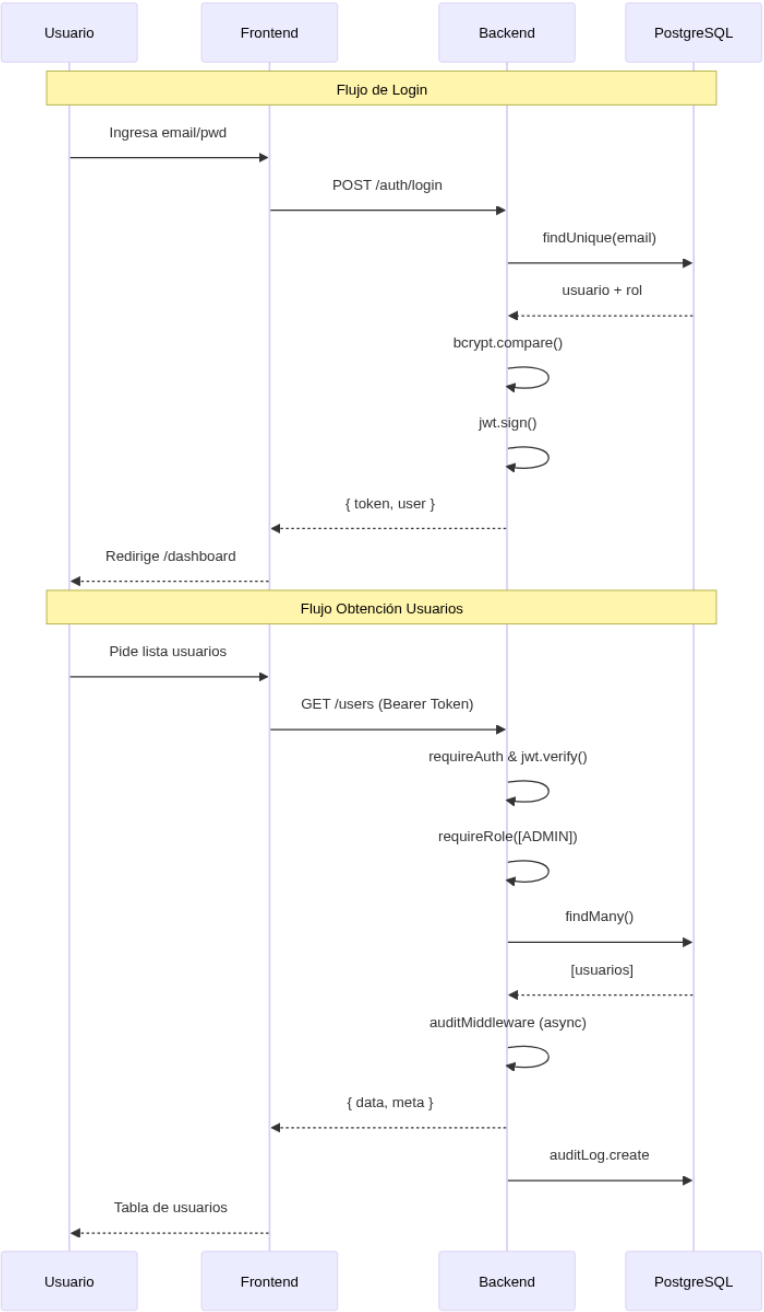
Se seleccionó una arquitectura en Capas para el backend, expuesto como API REST, combinado con un SPA React (Single-Page Application) como cliente.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      CLIENTE (Navegador / PWA)                       │
│                    React 18 + Vite · Puerto 5173                     │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐   │
│  │ Auth Module  │  │ Admin Module │  │  Domain Modules          │   │
│  │  Login/JWT   │  │ Users & Roles│  │  Obra · Compras · Inv.   │   │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘   │
│        │                │                        │                   │
│        └────────────────┴────────────────────────┘                   │
│                         │ HTTP / REST → /api/v1                      │
│                         │ Bearer JWT en cada request                 │
└─────────────────────────┼────────────────────────────────────────────┘
                          │
                          ▼
              ┌──────────────────────┐
              │   NGINX / Proxy Vite │  (proxy reverso en dev)
              └──────────────────────┘
                          │
┌─────────────────────────▼────────────────────────────────────────────┐
│                   BACKEND — Node.js + Express                        │
│                          Puerto 3001                                 │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                      CAPA DE MIDDLEWARES                     │   │
│  │  helmet · cors · morgan · auditMiddleware · requireAuth      │   │
│  │  requireRole (RBAC) · requireProjectAccess                   │   │
```

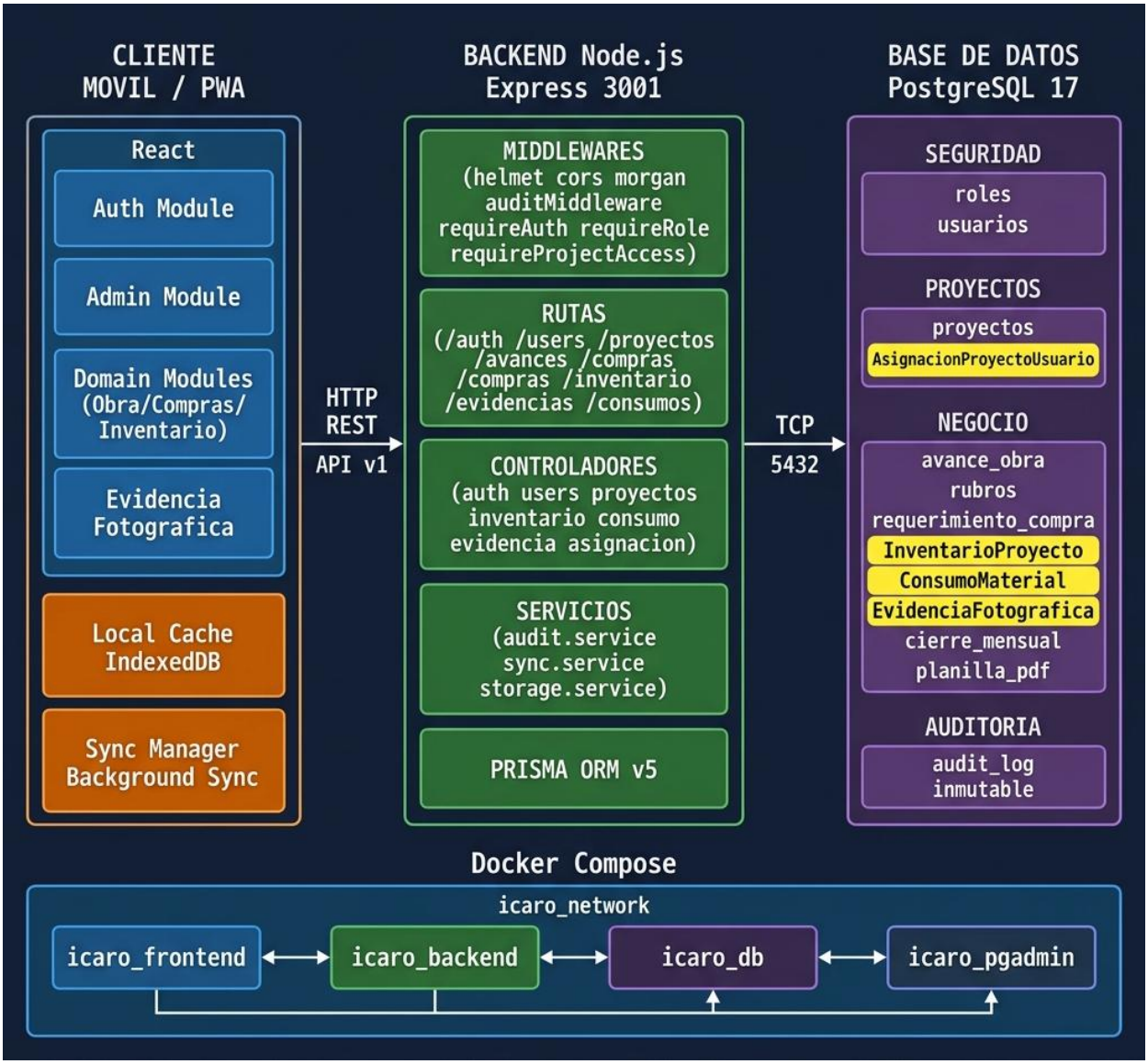
```
│   └──────────────────────────────────────────────────────────┘     │
│   ┌──────────────────────────────────────────────────────────┐     │
│   │                     CAPA DE RUTAS                        │     │
│   │  /auth · /users · /proyectos · /avances · /compras       │     │
│   └──────────────────────────────────────────────────────────┘     │
│   ┌──────────────────────────────────────────────────────────┐     │
│   │                   CAPA DE CONTROLADORES                  │     │
│   │  auth.controller · users.controller · proyectos.controller  │     │
│   └──────────────────────────────────────────────────────────┘     │
│   ┌──────────────────────────────────────────────────────────┐     │
│   │                    CAPA DE SERVICIOS                     │     │
│   │            audit.service · (futuro: email, storage)      │     │
│   └──────────────────────────────────────────────────────────┘     │
│   ┌──────────────────────────────────────────────────────────┐     │
│   │                  CAPA DE ACCESO A DATOS                  │     │
│   │              Prisma ORM · PrismaClient                   │     │
│   └──────────────────────────────────────────────────────────┘     │
└────────────────────────────────────────────────────────────────────┘
                            │ TCP 5432
                            ▼
┌────────────────────────────────────────────────────────────────────┐
│                  BASE DE DATOS — PostgreSQL 17                     │
│                                                                    │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐│
│   │ Seguridad    │  │ Proyectos    │  │         Negocio          │
│   │ roles        │  │ proyectos    │  │ avance_obra · rubro      │
│   │ usuarios     │  │ asignacion   │  │ requerimiento_compra     │
│   └──────────────┘  └──────────────┘  │ inventario_proyecto      │
│   ┌──────────────────────────────────►│ cierre_mensual · planilla_pdf │
│   │  audit_log (inmutable)            └──────────────────────────┘
│   └──────────────────────────────┘                                 │
```

```
└────────────────────────────────────────────────────────────────────┘
```

3. Diagrama de Capas y Flujo de una Petición



4. Diagrama Arquitectónico Completo (Componentes y Despliegue)



5. Patrones Arquitectónicos Adoptados

5.1 Patrón: Layered Architecture (Capas)

Descripción: El backend se organiza en 5 capas con dependencias unidireccionales (hacia abajo):

```
Middlewares → Rutas → Controladores → Servicios → Prisma (Datos)
```

Justificación:

- Cada capa tiene una única responsabilidad (SRP de SOLID)
- Permite testear capas de forma independiente (los tests de seguridad aíslan la capa de middlewares)

- El equipo puede extender un módulo sin tocar las otras capas

Alternativa descartada: Arquitectura Hexagonal (Ports & Adapters)

- Ofrece mayor desacoplamiento, pero su complejidad (interfaces explícitas, inversión de dependencias) es innecesaria para un equipo pequeño y un ERP con dominio bien definido.

5.2 Patrón: RBAC — Role-Based Access Control

Descripción: El acceso a recursos se controla mediante roles predefinidos, no por usuario individual.

```
Token JWT
    │
    ├─ requireAuth ─── Verifica firma y expiración
    │
    └─ requireRole([ROLES.ADMIN]) ── Verifica rol del payload
            │
        403 Forbidden si el rol
        no está en la lista.
```

Roles del sistema:

Rol	Módulos accesibles
Administrador del Sistema	Todos (gestión de usuarios, auditoría, config)
Presidente / Gerente	Proyectos, Avances, Compras, Reportes
Contador	Proyectos, Compras, Cierres, Reportes
Residente	Proyectos, Avances, Compras, Inventario
Auxiliar de Contabilidad	Compras, Cierres (soporte)
Bodeguero	Inventario, Movimientos

Justificación:

- Modelo estándar de seguridad en ERPs corporativos
- Fácil de auditar: saber qué rol hizo qué acción
- Escalable: agregar un nuevo rol solo requiere editar ROLES y MODULE_ROLES

Alternativa descartada: ABAC (Attribute-Based Access Control)

- Más granular, pero innecesariamente complejo para 6 roles definidos. ABAC requeriría una capa de evaluación de políticas que agrega latencia y deuda técnica sin beneficio real a esta escala.

5.3 Patrón: JWT Stateless Authentication

Descripción: La autenticación no requiere sesiones en servidor. El token JWT contiene el payload (id, email, rol, idRol) firmado con HS256.

```
[Login] ──▶ [Backend firma JWT] ──▶ [Cliente guarda en localStorage]
                                                  │
                                    [Cada request lo envía en Authorization]
                                                  │
                                    [Backend verifica firma, extrae payload]
                                    [No consulta DB para verificar la sesión]
```

Justificación:

- Sin estado en servidor: El backend puede escalar horizontalmente sin compartir sesiones
- Performance: La verificación del token es O(1) — solo operaciones criptográficas, sin DB
- Expiración incorporada: JWT soporta exp nativo; el middleware detecta TokenExpiredError automáticamente

Alternativa descartada: Sesiones con express-session + Redis

- Requeriría un servicio Redis adicional. Para < 50 usuarios concurrentes, el overhead es innecesario. JWT es una elección canónica y suficiente para este perfil de carga.

5.4 Patrón: Audit Log Inmutable (Append-Only)

Descripción: La tabla audit_log nunca recibe UPDATE ni DELETE. Solo se escribe con INSERT.

```
Operación CUD
     │
     ├─ Controlador ejecuta la operación principal
     │
     ├─ Response enviado al cliente (res.on('finish'))
     │
     └─ auditMiddleware → audit.service.logAction()
            │
        INSERT INTO audit_log (tabla, operacion, id_registro,
                               id_usuario, ip_origen, timestamp,
```

```
                                    datos_antes, datos_despues)
```

Justificación:

- Irreversibilidad: Ningún usuario, ni el Admin, puede modificar el historial
- Trazabilidad legal: En contratos de construcción pública, es requisito documentar quién aprobó cada requerimiento y cuándo
- Resiliencia: Si el log de auditoría falla, la operación principal ya fue procesada (no bloquea el flujo)

Alternativa descartada: Logs en archivos (Winston/Pino)

- Los logs de archivo son volátiles, difíciles de consultar y no relacionales. Un audit_log en PostgreSQL permite hacer JOIN con usuarios y proyectos para generar reportes de trazabilidad.

5.5 Patrón: API-First (REST con versionamiento)

El backend expone solo una API REST con prefijo /api/v1/. El frontend es un cliente más (como podría serlo una app móvil futura).

```
/api/v1/auth/*       ← Autenticación pública
```

```
/api/v1/users/*      ← Gestión de usuarios (Admin)
/api/v1/proyectos/*  ← RBAC + Control por fecha
/api/v1/avances/*    ← (futuro sprint)
```

```
/api/v1/compras/*    ← (futuro sprint)
```

Justificación:

- Desacoplamiento total: El frontend puede cambiarse por una app React Native sin tocar el backend
- Versionamiento: /v1/ permite introducir breaking changes en /v2/ sin afectar clientes existentes
- Testeable: La suite de pruebas con Supertest puede probar el backend de forma independiente al frontend

5.6 Selección de Base de Datos: PostgreSQL 17

Justificación vs alternativas:

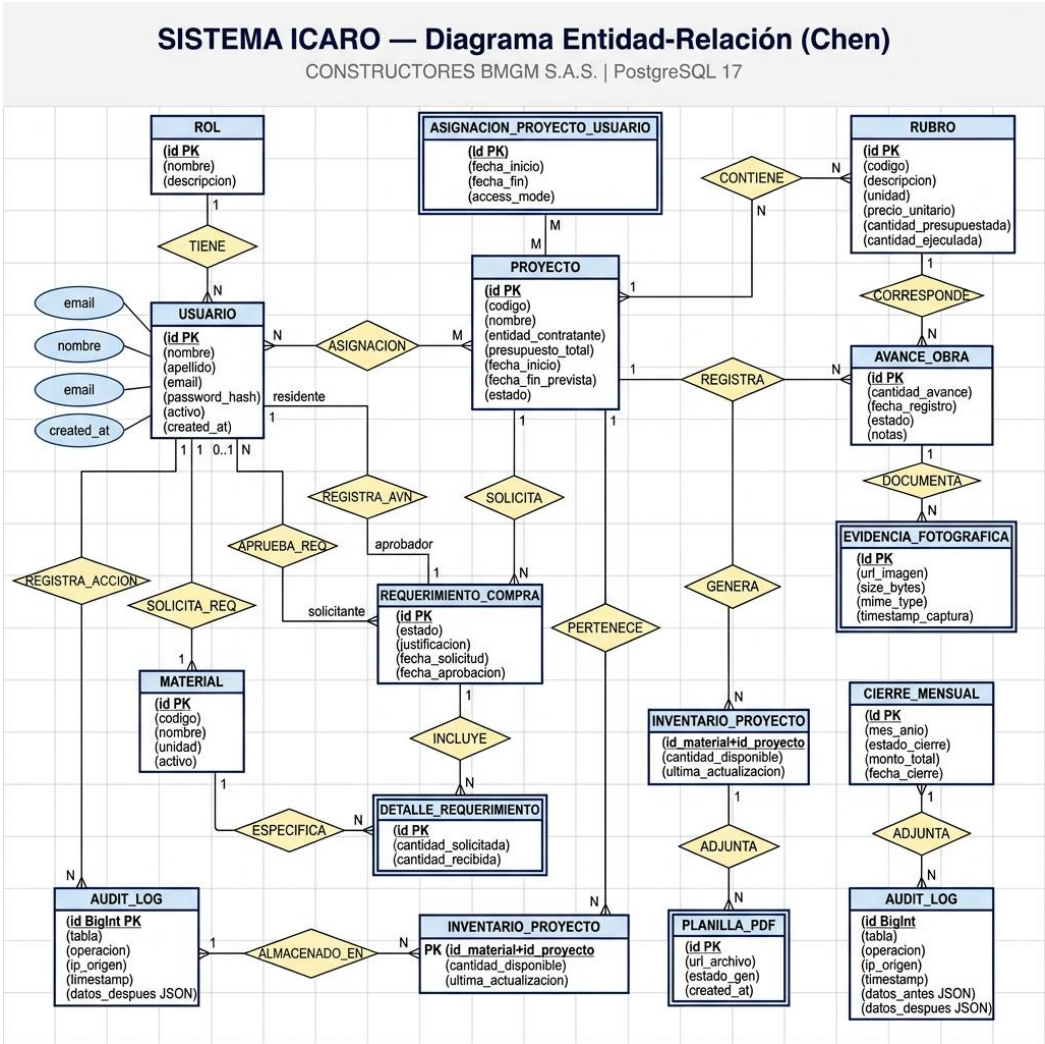
Criterio	**PostgreSQL 17**	**MySQL 8**	**MongoDB**	**SQLite**
Integridad referencial	Excelente (FK, checks, constraints)	Buena	No nativa	Buena

Soporte JSON/JSONB	Excelente (audit_log datosAntes/despues)	Limitado	Nativo	Limitado
Transacciones ACID	Total	Total	Parcial (desde v4)	Total
Funciones/triggers	Completo	Limitado	No aplica	Limitado
Escalabilidad	Alta (particionado, replication)	Alta	Muy alta	Solo local
Tipado estricto	Muy estricto	Estricto	Dinámico	Dinámico

PostgreSQL es la elección correcta porque:

1. El dominio del ERP tiene relaciones complejas entre proyectos, rubros, usuarios y auditoría
2. Los campos audit_log.datosAntes y datosDespues usan JSONB, que PostgreSQL maneja nativamente con indexado
3. La integridad referencial es crítica: un avance de obra debe siempre apuntar a un proyecto y rubro existentes
4. El volumen de datos (< 10 millones de registros estimados) es ideal para un RDBMS, no requiere NoSQL

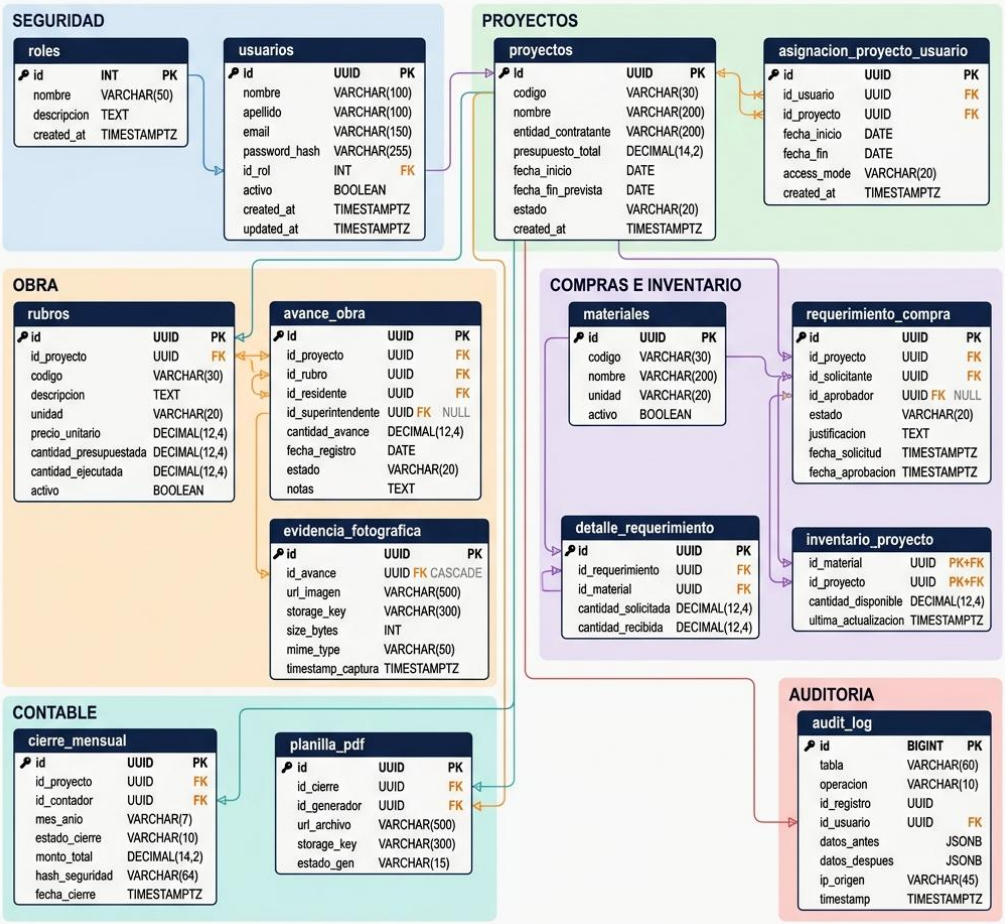
5.6.1. Modelo base de datos: ER



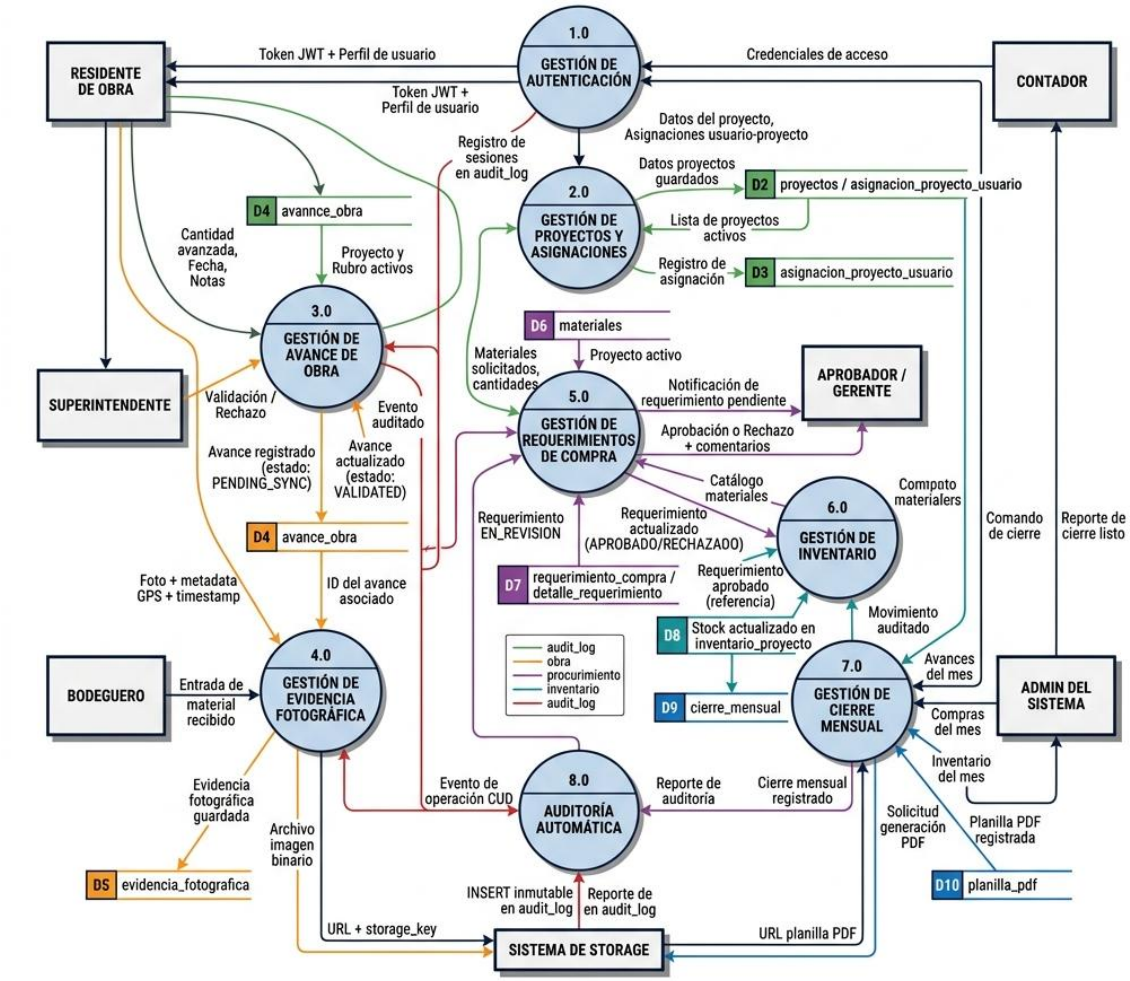
5.6.2. Modelo de base de datos: **Esquema relacional**

SISTEMA ICARO — Esquema Relacional

PostgreSQL 17 | Prisma ORM v5 | 14 tablas



5.6.3 Flujo de datos:



5.7 Selección de ORM: Prisma v5

```
Prisma ORM
│
├─ schema.prisma ─── Fuente única de verdad del esquema
│
├─ prisma migrate ── Migraciones versionadas con Git
│
├─ PrismaClient ──── Type-safe queries (autocompletion)
│
└─ prisma seed ───── Datos iniciales reproducibles
```

Justificación vs alternativas:

Criterio	Prisma v5	TypeORM	Sequelize	Knex
Type safety	Excelente	Buena	Limitada	No
DX (Developer Experience)	Excelente	Media	Media	Baja
Migraciones automáticas	Sí	Sí	Sí	Manual
Curva de aprendizaje	Baja	Alta	Media	Alta
Soporte PostgreSQL JSONB	Sí	Parcial	Parcial	Sí

6. Patrón de Despliegue: Docker Compose

```
┌──────────────────────────────────────────┐
│  docker-compose.yml                      │
│                                          │
│  services:                               │
│    db:       postgres:17-alpine          │  ← Base de datos aislada
│    backend:  Node.js (Dockerfile)        │  ← Imagen reproducible
│    frontend: React/Vite (Dockerfile)     │  ← Build consistente
│    pgadmin:  dpage/pgadmin4              │  ← Solo desarrollo
│                                          │
│  networks: icaro_network (bridge)        │  ← Aislamiento de red
│  volumes:  icaro_postgres_data           │  ← Persistencia de datos
└──────────────────────────────────────────┘
```

Ventajas de usar Docker Compose para este proyecto:

Criterio	Docker Compose	Kubernetes	Railway/Render	VPS Manual
Complejidad de setup	Baja	Muy alta	Media	Alta
Reproducibilidad	Total	Total	Media	Baja
Costo operacional	Bajo	Alto	Medio	Medio
Curva de aprendizaje	Baja	Muy alta	Baja	Alta
Ideal para	Dev + staging	Prod multi-nodo	SaaS simples	Proyectos legacy

Kubernetes es innecesario para < 50 usuarios concurrentes y un equipo de desarrollo pequeño. Docker Compose garantiza ambientes idénticos entre desarrollo y producción sin complejidad adicional.

7. Frontend: PWA con React + Vite

Justificación del Stack

Criterio	React 18 + Vite	Next.js	Angular	Vue.js
Modo PWA (offline)	Sí (vite-plugin-pwa)	Sí	Sí	Sí
Tamaño del bundle	Pequeño (Vite HMR)	Mediano	Grande	Pequeño
Curva aprendizaje	Media	Media-Alta	Alta	Baja
Ecosistema	Muy amplio	Muy amplio	Corporativo	Amplio
SSR necesario	No (ERP login-required)	Innecesario	Innecesario	Innecesario

Next.js fue descartado porque su valor principal (SSR/SSG para SEO) no aplica a un ERP corporativo donde todas las páginas requieren autenticación previa.

Patrón: Context API + Axios Interceptors

```
AuthContext (React Context)
    │
    ├─ user: { id, nombre, rol }
    ├─ isAuthenticated: boolean
    ├─ login() / logout()
    │
    └─ Se inyecta en toda la app vía <AuthProvider>

Axios Instance (utils/axios.js)
    │
    ├─ Interceptor request: añade 'Authorization: Bearer <token>'
    │
    └─ Interceptor response: captura 401 → fuerza logout + redirect /login
```

9. Estructura de Archivos del Proyecto

```
Sistema_ICARO/
├── docker-compose.yml              ← Orquestación de servicios
│
├── backend/
│   ├── src/
│   │   ├── server.js               ← Entry point + middlewares globales
│   │   ├── middlewares/
│   │   │   ├── auth.middleware.js  ← requireAuth, requireRole, ROLES
│   │   │   ├── audit.middleware.js ← Auditoría automática CUD
│   │   │   └── projectAccess.middleware.js ← Control por proyecto y fecha
│   │   ├── controllers/
│   │   │   ├── auth.controller.js  ← Login, me, recover-password
│   │   │   └── users.controller.js ← CRUD de usuarios
│   │   ├── routes/
│   │   │   ├── auth.routes.js
│   │   │   ├── users.routes.js     ← Solo ADMIN
│   │   │   └── proyectos.routes.js ← requireProjectAccess
│   │   ├── services/
│   │   │   └── audit.service.js    ← logAction, logFromRequest, extractIp
│   │   └── utils/
│   │       └── jwt.js              ← generateToken, verifyToken
│   ├── prisma/
│   │   ├── schema.prisma           ← 14 modelos, fuente única de verdad
│   │   ├── seed.js                 ← Datos iniciales reproducibles
│   │   └── migrations/             ← Historial de cambios de esquema
│   └── tests/
│       └── security.test.js        ← 15 pruebas Jest + Supertest
│
└── frontend/
    └── src/
        ├── store/
        │   └── AuthContext.jsx     ← Estado global de sesión
        ├── services/
        │   └── usersApi.js         ← API service + fallback mock
        ├── utils/
        │   └── axios.js            ← Interceptors JWT + 401-handler
        ├── pages/                  ← Enrutamiento React Router
        └── views/
            └── admin/
                └── AdminUsersPermissionsView.jsx ← CRUD usuarios
```

10. Decisiones Arquitectónicas Rechazadas

Alternativa	Por qué fue rechazada

Microservicios	Con un equipo pequeño y < 50 usuarios, la complejidad operacional (service discovery, distributed tracing, inter-service auth) supera los beneficios. El monolito modular es suficiente y más mantenible.
GraphQL	El modelo de datos es relacional y bien definido. REST con endpoints específicos es más predecible, más fácil de testear y más familiar para el equipo.
MongoDB	Las relaciones complejas entre proyectos, rubros, usuarios y auditoría requieren integridad referencial estricta. Un documento store impondría validaciones manuales costosas.
Sesiones con Redis	JWT stateless es suficiente para < 50 usuarios concurrentes. Redis añade un servicio extra, complejidad de configuración y un punto más de fallo sin beneficio medible.
Next.js SSR	El ERP es 100% privado (requiere login). SSR/SSG no aporta valor sin SEO público. React + Vite SPA es más simple y eficiente para este caso.
Kubernetes	Innecesario para la escala actual. Docker Compose garantiza reproducibilidad con una curva de aprendizaje mínima.